Lab 3 - Requirements Engineering

Student Sheet

Software Web and Mobile Engineering Duration: 100 Minutes

Goal

Last week you worked from a short client request. This time you have notes from several people who want different things from the same product. Your job is to turn those notes into a requirements document that another team could design and test.

The product is CampusPulse, a small social network for the university community. You will map the stakeholders, decide the boundary of the first release, write user and system requirements, add user stories and priorities, and then check the document for problems.

There is no program code and no notebook. Work in the three Markdown templates in `lab03_files/`. Commit after every task and push the finished work, as in Lab 2. The deliverables and marks are on the last page.

Words From Lecture 2 You Will Use

Word	Meaning in one line
Stakeholder	Anyone who affects the system, is affected by it, or can stop it succeeding.
Power-interest grid	A way to decide how closely each stakeholder should be involved.
User requirement	A customer-readable statement of a service or need.
System requirement	A precise statement for the people who will design, build, and test the system.
Functional [FR]	What the system must do.
Non-functional [NFR]	How well it must behave, stated with a measure and a condition.
User story	A need written from one stakeholder's point of view, with the reason it matters.
Acceptance criterion	A checkable result that decides whether a story is complete.
Traceability	The links from stakeholder evidence to requirements and stories.
Validation	Checking that the specification is the right one, not merely a well-written one.

Setup

Download `lab03_files.zip` from the LMS. It contains this sheet and three Markdown templates. In the terminal:

```
cd ~/Downloads
unzip lab03_files.zip
mv lab03_files ~/ai1220/lab03
cd ~/ai1220
git switch -c lab03-requirements
git add lab03 && git commit -m "Start lab03 from templates"
```

Keep the headings in the templates. They show where each part of the work goes.

Commit after every task. The final files matter, but the history should also show how the specification developed and where validation changed it.

Source Notes: CampusPulse

Student Affairs is considering a browser-based service for campus events and announcements. These notes come from a short meeting and a few follow-up messages. They are evidence, not requirements ready to copy.

S1 - Student attendee. "I follow several clubs and I am tired of checking email, Instagram, and group chats. If I RSVP, do not put my name on a public list unless I choose that. I normally use my phone; some students use screen readers."

S2 - Group officer. "One officer may start an announcement and another finish it, but only approved officers should publish. Some events are for the whole university and some are members-only. If the place or time changes, tell the people who RSVP'd."

S3 - Campus moderator. "A report must show what was reported and why. I may need to hide an event immediately, but I still need the evidence if the group appeals. We also need a record of who made the decision."

S4 - Student Affairs. "An official badge must mean that we checked the group. The pilot is for 5,000 students and 200 groups before Orientation Week."

S5 - Data Protection Officer. "Collect only the personal data this service needs. RSVP lists should start private. If an event is cancelled, delete its attendance data within 30 days."

S6 - Abuse cases. Someone may imitate a verified group and publish a phishing event. A compromised group account may post the same announcement repeatedly.

The first release uses university sign-in and has no native mobile app. It covers verified groups, announcements and events, following, RSVP, audience visibility, corrections, reports, moderation, and appeals. Direct messages, external users, payments, video hosting, and AI recommendations are out. The brief gives no peak traffic figure for Orientation Week.

Task 1: Stakeholders and Conflicts 20%

In `stakeholders.md`, complete S1–S6. For each one, name the stakeholder type and power-interest quadrant, then write its main goal, concern, and how you would involve or monitor it. Do not put everyone in the same quadrant.

Describe at least two conflicts. Name both sources, explain what pulls in opposite directions, and either make a decision or write the exact question you would ask next.

Task 2: Scope and User Requirements 20%

In Sections 1–2 of `REQUIREMENTS.md`, list at least three things that are in the first release and at least two that are out. Then write at least five user requirements in this form:

```
- UR-1 [Must] ... [Source: S1]
```

Use one need per line. These are for a customer to read, so do not begin them with "The system shall" and do not describe an implementation.

Task 3: System Requirements 25%

In Sections 3–4, write at least six functional requirements and four non-functional requirements. Every one must trace to an existing UR.

```
- FR-1 [Must] The system shall ... [Source: UR-1]
- NFR-1 [Must] The system shall ... [Measure: ...] [Source: UR-1]
```

An FR describes behaviour that a tester can observe. An NFR names what is measured, the target, and the condition. If the brief does not supply a number, record your number as an assumption or leave it as an open question. Do not make it look as if a stakeholder supplied it.

Task 4: Stories, Priorities, and Traceability 25%

In Sections 5–7, write at least three stories from different stakeholder viewpoints. Each needs two acceptance criteria. Across the set, include at least one failure, permission boundary, privacy rule, or policy case.

Fill all four MoSCoW categories, including a real Won't list. Add at least four rows to the traceability table. Each row must connect a stakeholder need to a UR, an FR or NFR, and a story. If the link is not real, change the document rather than filling the table for appearance.

Task 5: Validate and Revise 10%

In `VALIDATION.md`, answer the five questions from Lecture 2: validity, consistency, completeness, realism, and verifiability. Refer to your own IDs and give evidence; "yes" is not an answer.

Then choose one requirement that needs work. Record its old wording, the problem, and the revised wording. Make the same change in `REQUIREMENTS.md` and in the traceability table.

Submit: Merge and Push

Merge the branch into `main` with a merge commit and push. Check on GitHub that `lab03/` contains the three completed Markdown files. Submission is the pushed repository at the end of the session.

Deliverables

All inside `lab03/` in your `ai1220` repository, pushed before the end of the session.

#	Deliverable	Mark
1	`stakeholders.md`: six analyses and at least two conflicts	20%
2	`REQUIREMENTS.md`: scope and at least five traced URs	20%
3	`REQUIREMENTS.md`: at least six FRs and four measurable NFRs	25%
4	`REQUIREMENTS.md`: three stories, MoSCoW, four trace rows	25%
5	`VALIDATION.md`: five checks and one real revision	10%
	Branch merged into `main`; work shown in at least five commits	gate

The last row is not marked but gates the rest: work that never reached `main` on GitHub before the end of the session counts as not submitted.

Before You Submit

- The scope and the Won't list agree.
- Every FR and NFR points to a UR that exists.
- Every NFR has a measure and a condition.
- Every acceptance criterion can pass or fail.
- Every traceability row describes a link you can explain.
- The revised requirement is updated everywhere it appears.

Read More After the Lab

- Sommerville, *Software Engineering*, ch. 4 - requirements engineering: `https://software-engineering-book.com/`
- Mountain Goat Software on user stories: `https://www.mountaingoatsoftware.com/agile/user-stories`
- Agile Business Consortium on MoSCoW prioritisation: `https://www.agilebusiness.org/dsdm-project-framework/moscow-prioririsation.html`